

AULA EXPLORATÓRIA PRÁTICA · CAPÍTULO 4
PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Roteiro de aula exploratória prática

Capítulo 4 — Análise léxica e análise sintática em Java

5 min

Preparar

17 min

Léxico

9 min

Sintaxe

23 min

Parsers

6 min

Entregar

NOME	Heitor Henrique Scramim de Freitas
RA	24190382-2
TURMA	A
DATA	31/08/2026

PREPARAÇÃO · 5 MINUTOS

Sua missão em 60 minutos

Nesta atividade, você vai **copiar, executar, alterar e registrar** pequenos programas Java. Os programas tornam visíveis etapas que normalmente ficam escondidas dentro de um compilador.

Regra principal: não pesquise durante a atividade. Siga cada passo na ordem, altere somente o que for solicitado e use a saída do programa para observar o conceito.

Antes de começar

1. **Abra o compilador.** Use um ambiente com Java 17 ou superior, no computador ou no navegador.
2. **Localize os arquivos.** Os cinco programas estão separados em pastas numeradas de 01 a 05.
3. **Preencha sua identificação.** Em cada programa, substitua os valores de ALUNO e RA antes da primeira execução.
4. **Mantenha a tela visível.** Ao tirar um print, mostre simultaneamente uma parte do código e o console.
5. **Não avance com erro inesperado.** Na Estação 3 os erros são propositais; nas demais, confira se você copiou o programa inteiro.

Evidências da atividade

Ao final: você terá seis prints. O nome exato de cada arquivo aparece na estação correspondente e será conferido na seção final.

1

ESTAÇÃO 1

De caracteres a lexemas e tokens

Observe como uma linha de código-fonte é dividida em unidades com significado.

Tempo	9 minutos
Arquivo	01-lexemas-tokens/Main.java
Evidência	01_tokens.png

O que fazer

1. Abra o arquivo da Estação 1 e copie todo o conteúdo para o compilador.
2. Substitua ALUNO e RA pelos seus dados. Use somente letras e números dentro das aspas.
3. Execute o programa sem alterar a entrada inicial.
4. Leia no console as colunas LEXEMA e TOKEN. Localize um identificador, um operador, um número inteiro e o ponto e vírgula.
5. Troque somente o valor da constante ENTRADA por uma atribuição personalizada.

```
static final String ENTRADA = "total_mg = valor + 25;";  
// Troque mg por suas iniciais e 25 pelos dois últimos dígitos do RA.
```

6. Execute novamente e confira se o novo identificador e o novo número aparecem na lista.
7. Tire o print com a linha ENTRADA, sua identificação e o console visíveis.
8. Salve a imagem como 01_tokens.png.

Observe

Conclusão guiada: o texto `total_mg` é um lexema. IDENTIFICADOR é a categoria, ou token, atribuída a esse lexema.

- ☒ Encontrei pelo menos dois lexemas classificados como IDENTIFICADOR.
- ☒ Percebi que lexema e token não são a mesma coisa.

2

ESTAÇÃO 2

O que o analisador léxico reconhece

Compare espaços, comentários, palavras reservadas, identificadores e caracteres inválidos.

Tempo	8 minutos
Arquivo	02-classes-reservadas/Main.java
Evidência	02_lexico.png

O que fazer

1. Abra o arquivo da Estação 2, preencha ALUNO e RA e execute o programa.
2. Compare os testes A e B. A segunda entrada possui mais espaços e um comentário, mas os tokens essenciais devem permanecer equivalentes.
3. No teste C, localize `int` e `while`. Depois localize `inteiro` e `intValor`.

4. Substitua o valor de IDENTIFICADOR_PERSONALIZADO por um nome formado com suas iniciais.

```
static final String IDENTIFICADOR_PERSONALIZADO = "nota_mg";  
// Exemplo: nota_mg. Não use espaços nem acentos.
```

5. Execute novamente. Confira que `int` e `while` aparecem como PALAVRA_RESERVADA e que seu nome aparece como IDENTIFICADOR.
6. No teste D, localize a mensagem ERRO_LEXICO referente ao caractere #.
7. Tire um print mostrando seu identificador personalizado e a parte do console com PALAVRA_RESERVADA, IDENTIFICADOR e ERRO_LEXICO.
8. Salve a imagem como 02_lexico.png.

Observe

- ☒ Espaços adicionais não criaram novos tokens.
- ☒ O comentário foi ignorado pelo analisador.
- ☒ `int` e `inteiro` foram classificados de maneira diferente.
- ☒ O caractere # foi rejeitado pelo analisador didático.

Ligação com os slides: o programa agrupa letras e dígitos em classes de caracteres e consulta uma tabela para decidir se um nome é uma palavra reservada.

3

ESTAÇÃO 3

Quando o compilador encontra erros

Use os diagnósticos do compilador para reparar um programa Java curto.

Tempo	9 minutos
Arquivo	03-diagnosticos/Main.java
Evidências	03_erro_compilador.png e 04_programa_corrigido.png

Importante: este arquivo foi entregue com erros propositalmente. Nesta estação, falhar na primeira compilação significa que o experimento está funcionando.

Primeira compilação

1. Abra o arquivo, preencha ALUNO e RA e tente compilar sem corrigir as linhas marcadas.
2. Leia a primeira mensagem de erro. Procure o número da linha e algum símbolo esperado pelo compilador.
3. Sem corrigir ainda, tire um print com o código e a mensagem do compilador.
4. Salve a imagem como 03_erro_compilador.png.

Reparo mecânico

1. Procure as linhas marcadas com CORRIGIR 1, CORRIGIR 2 e CORRIGIR 3.
2. Acrescente o ponto e vírgula que está faltando.
3. Feche corretamente o parêntese da condição.
4. Feche corretamente o texto entre aspas.
5. Compile novamente. Se surgir outro erro, volte para a linha indicada e confira somente os sinais de pontuação.
6. Quando o programa executar, tire um novo print e salve como 04_programa_corrigido.png.

Classificação rápida

- ☒ Texto entre aspas não finalizado: problema na formação de um elemento léxico.
- ☒ Ponto e vírgula ou parêntese ausente: estrutura sintática incompleta.
- ☒ Depois das correções, o programa foi aceito e executado.

4

ESTAÇÃO 4

Parser descendente recursivo

Acompanhe as chamadas de `expr`, `term` e `factor` enquanto o parser percorre a expressão.

Tempo	14 minutos
Arquivo	04-parser-descendente/Main.java
Evidência	05_descendente.png

Execução válida

1. Abra o arquivo, preencha ALUNO e RA e execute a entrada original $2 + 3 * 4$.
2. No console, acompanhe as mensagens ENTRANDO e SAINDO para `expr`, `term` e `factor`.
3. Observe que a multiplicação é tratada dentro de `term` antes de `expr` terminar a soma.
4. Troque a constante ENTRADA pela expressão personalizada abaixo.

```
static final String ENTRADA = "(5 + 3) * 4";  
// Troque 5 pelo último dígito não nulo do seu RA.
```

5. Execute e procure a mensagem ENTRADA ACEITA.
6. Tire um print mostrando a expressão, sua identificação e parte do rastreamento.
7. Salve a imagem como 05_descendente.png.

Execução inválida e observação extra

1. Troque temporariamente a entrada por $2 + * 4$ e execute. Localize o token que estava no *lookahead* quando ocorreu o erro.
2. Restaure uma entrada válida, altere DEMONSTRAR_RECURSAO_ESQUERDA para `true` e execute uma vez. Depois volte o valor para `false`.

Conclusão guiada: o parser descendente começa pela regra mais geral e chama métodos associados aos não terminais. Uma regra que chama a si mesma antes de consumir entrada pode repetir indefinidamente.



Percebi que os parênteses e o *lookahead* orientaram o reconhecimento.



Observei por que a recursão à esquerda é inadequada para esse parser.

5

ESTAÇÃO 5

Parser ascendente: deslocar e reduzir

Observe a entrada ser transferida para uma pilha e reduzida até o símbolo inicial.

Tempo	9 minutos
Arquivo	05-parser-ascendente/Main.java
Evidência	06_ascendente.png

O que fazer

1. Abra o arquivo, preencha ALUNO e RA e execute a entrada `id+id*id`.
2. Leia as três colunas: PILHA, ENTRADA e ACAO.
3. Localize uma ação SHIFT. Nela, um item da entrada é deslocado para a pilha.
4. Localize uma ação REDUCE. Nela, uma sequência reconhecida é substituída pelo lado esquerdo de uma regra.
5. Localize a ação ACCEPT e tire um print com a identificação e as três colunas visíveis.
6. Salve a imagem como `06_ascendente.png`.
7. Troque a entrada por `id+*id`, execute novamente e localize a ação ERROR. Não é necessário tirar outro print.

```
static final String ENTRADA_VALIDA = "id+id*id";  
static final String ENTRADA_INVALIDA = "id+*id";
```

Compare as duas direções

- ☒ Descendente: parte de expr e avança em direção aos componentes menores.
- ☒ Ascendente: parte dos tokens e realiza reduções até chegar ao símbolo inicial.
- ☒ SHIFT movimenta a entrada; REDUCE aplica uma regra; ACCEPT conclui a análise.

Limite do experimento: a tabela LR já está pronta dentro do programa. Nesta aula você observa seu uso, mas não precisa construir as partes ACTION e GOTO.

CONFERÊNCIA FINAL · 6 MINUTOS

Organize as evidências antes de entregar

Use os minutos finais para conferir nomes, conteúdo visível e ordem dos arquivos.

Checklist dos arquivos

- ☒ 01_tokens.png mostra a entrada personalizada e a lista de tokens.
- ☒ 02_lexico.png mostra palavra reservada, identificador e erro léxico.
- ☒ 03_erro_compilador.png mostra uma tentativa que falhou.
- ☒ 04_programa_corrigido.png mostra a execução depois dos reparos.
- ☒ 05_descendente.png mostra expr, term ou factor no rastreamento.
- ☒ 06_ascendente.png mostra PILHA, ENTRADA, ACAO e ACCEPT.

O que cada print deve mostrar

- ☒ Minha identificação está visível no código ou no console.
- ☒ A alteração pedida na estação está visível.
- ☒ O console ou a mensagem do compilador pode ser lido.
- ☒ Não cortei o trecho principal da evidência.
- ☒ Usei exatamente os nomes de arquivo indicados.

Entrega: cole os seis prints na folha de evidências fornecida pelo professor. Não escreva uma explicação longa: marque apenas as verificações solicitadas e entregue no formato definido para a turma.

O que você acabou de observar

Síntese: o analisador léxico transforma caracteres em tokens; o parser verifica a gramática. Rastreamentos e pilhas mostram que as abordagens descendente e ascendente percorrem a estrutura em direções diferentes.

Referência conceitual: SEBESTA, Robert W. *Conceitos de Linguagens de Programação*. 11. ed. Capítulo 4: análise léxica e sintática.

Roteiro contínuo — sem divisão em páginas ou quebras obrigatórias entre as estações.